

•

•

•

816,654,829 individual census records from 1850 to 1950.

YEAR	SAMPLE	SERIAL	STATEICP	COUNTYICP	COUNTYICP	PERNUM	FAMSIZE	MOMLOC	POPLOC	SPLOC	NCHILD	RELATE	SEX	AGE	BIRTHYR	RACE	BPL
1850	185002	1	41	10	1	3	0	0	0	0	1	1	2	57	1793	1	13
1850	185002	1	41	10	2	3	1	0	0	0	0	3	2	14	1836	1	1
1850	185002	1	41	10	3	3	0	0	0	0	0	9	1	7	1843	1	1
1850	185002	2	41	10	1	5	0	0	2	3	1	1	31	1819	1	1	
1850	185002	2	41	10	2	5	0	0	1	3	2	2	25	1825	1	1	
1850	185002	2	41	10	3	5	2	1	0	0	3	2	8	1842	1	1	
1850	185002	2	41	10	4	5	2	1	0	0	3	2	6	1844	1	1	
1850	185002	2	41	10	5	5	2	1	0	0	3	2	4	1846	1	1	
1850	185002	3	41	10	1	7	0	0	2	5	1	1	46	1804	1	13	
SERIAL (Household ID)	A code that provides a unique identifier for the household itself.																
POPLOC (Father Location)	A code that tells you the exact line number on the census page where a specific person's father is located.																
MOMLOC (Mother Location)	A code that indicates the exact line number on the census page where a person's mother is located.																
SPLOC (Spouse Location)	A structural pointer used to indicate the location of the person's spouse .																
FBPL	The father's birthplace.																
MBPL	The mother's birthplace																
BPL	Birth place indicates the U.S. state, the outlying U.S. area or territory, or the foreign country where the person was born.																
RELATE	Relationship to household head																
STATEICP	Code of the state of residence																
COUNTYICP	Code of the county of residence																
NCHILD	Number of children in the residence.																
PERNUM	Number ID of the person in the residence																
SAMPLE	The first four digits are the year of the census/survey;The next two digits identify the sample within the year.																
Number of lines in the census file			816,654,829		(census records)												
(1850-1950)																	

Process step 2. The Ingestion Engine (DatabaseVault.py)

-

Input: Raw .csv files (census data).

-

Process: Uses a Python ThreadPoolExecutor to parallelize file reading. Bypasses matching logic completely to ensure safe ingestion. Trims long strings with many blanks at the end of item. Cleans standard variables via the JSON codebook while preserving raw composite IDs (Sample + Serial + Pernum).

-

Output: Pristine, read-only SQLite vaults: MasterVault_ALL.db (the 100% data) and MasterVault_ALLs.db (the 5% samples with restored names).

Note: the samples database is smaller and easy to view in the DB browser.

Process step 3. Phase 1: The AI Anchors (run_analyst.py)

-

Inputs: MasterVault_ALL.db, MasterVault_ALLs.db, GedcomVault.db (your family GEDCOM), and DeathIndexVault.db. (BRILS database downloaded from the internet.)

-

Process: Uses DuckDB to perform an in-memory "Squash" (merging the 5% sample names onto the nameless 100% records). Filters out the nameless masses. Uses the Splink 4 AI library to perform probabilistic string matching (Jaro-Winkler, Levenshtein) on the named individuals to account for historical typos. Mints collision-proof 16-character UUIDs via the identity registry.

-

Output: ~15.5 million named, de-duplicated Golden Records locked safely into CleanVault.db.

Note: The **BRILS** (Beneficiary Identification Records Locator Subsystem) database is a massive historical index of deceased U.S. military veterans and their dependents who received benefits from the Department of Veterans Affairs. It contains over 19 million records useful for genealogical and historical research.

The Thomas Askey Family GEDCOM is 1,500 family members that have been gathered using Ancestry.com over the last twenty years.

Process step 4. Phase 1a: The Living Room Sweep (ExpandHouseholds.py)

- Inputs: CleanVault.db (The Anchors) and MasterVault_ALL.db (The Raw Data).
 - Process: Relational graph traversal. Looks at the specific households containing Golden Records, traverses back to the raw database, and uses POPLOC (Father) and MOMLOC (Mother) pointers to locate their unnamed children and spouses. Mints auto-generated IDs for these dependents.
 - Output: Inserts ~4.5 million newly identified dependent Golden Records into CleanVault.db.

The Golden Records

In the Askey Genealogy Data Engine, a Golden Record represents a single, mathematically verified human being distilled from decades of messy, error-prone historical census data. They are initially minted by the Splink AI engine, which uses probabilistic string matching to resolve 19th-century typos, and are further expanded through relational graph traversal (POPLOC/MOMLOC) to identify anonymous dependents. Stored exclusively in the pristine CleanVault.db, each Golden Record acts as an immutable anchor for the family tree. It is assigned a permanent 16-character cryptographic UUID (the "St. Joe's ID") and stores a composite list of vault_pointers (e.g., 1850_1234_1|1860_5678_2). This architecture elegantly separates the resolved identity "skeleton" from the raw historical "muscle," allowing the system to instantly map a single person back to their exact line numbers in the raw MasterVault_ALL.db to generate a perfectly cited GEDCOM file. It covers the what (a verified human), the "how" (Splink AI and graph math), and the "why" (separating the clean skeleton from the raw data to build the GEDCOM).

Process step 5. Phase 1a: The Time Machine (LinkFamiliesByDemographics.py)

-
- Inputs: CleanVault.db (Anchored Households) and MasterVault_ALL.db.
-
- Process: Bridges nameless families across decades using a 5-step DuckDB query. Projects Head/Spouse ages +10 years using high-speed Range Joins (BETWEEN). Uses FBPL (Father's Birthplace) and MBPL (Mother's Birthplace) as strict cryptographic filters to mathematically prevent Cartesian explosions.
-
- Output: Successfully linked multi-decade family vectors saved into DemographicMatches.db.

Process step 6. Phase 1a: The Universal Web (AssignFamilyIDs.py)

-
- Inputs: CleanVault.db and MasterVault_ALL.db.
-
- Process: Implements a high-speed Union-Find (Disjoint Set) graph algorithm in Python. Connects intra-decade households via IPUMS SERIAL & FAMUNIT, then binds them across decades using the Golden Record vault_pointers.
-
- Output: Maps the entire country into unique FAM_... buckets, stored in the universal_families table inside CleanVault.db.

Process step 7. The Payoff (export_gedcom.py)

-

Inputs: CleanVault.db and MasterVault_ALL.db.

-

Process: Reads the Golden Records and translates their vault_pointers back into physical historical events (Census Years, Ages, Locations). Groups individuals logically by families based on the relational tables. Applies strict GEDCOM 7.0.18 formatting, appending permanent St. Joe's IDs as REFN tags for easy software searching.

-

Output: A massive, fully-cited census_all_export.ged file ready to be imported into Family Tree Maker, Ancestry, or RootsMagic.

This flow gives you the exact skeleton of your architecture: separating the heavy lifting (DuckDB), the probabilistic logic (Splink), and the relational graph math (Python) into highly observable steps. Let me know if you need any adjustments to these summaries for the boxes on your diagram!